```
likelihood = gpytorch.likelihoods.GaussianLikelihood(
    noise_constraint=gpytorch.constraints.GreaterThan(1e-1) ❶
)
```

❶ The constraint forces the noise to be at least 0.1.

8. We train the GP model implemented above using gradient descent as follows:

```
model = GPModel(train_x, train_y, likelihood)

optimizer = torch.optim.Adam(model.parameters(), lr=0.01) ❶
mll = gpytorch.mlls.ExactMarginalLogLikelihood(likelihood, model) ❷

model.train() ❸
likelihood.train() ❸

for i in tqdm(range(10)):
    optimizer.zero_grad()

    output = model(train_x)
    loss = -mll(output, train_y)

    loss.backward()
    optimizer.step()
```

❶ The gradient descent optimizer Adam.

❷ The (negative) marginal log likelihood loss function.

❸ Enable training mode.

The total training time was 24 seconds on a Macbook.

9. We implement the variational GP model as follows:

```
class ApproximateGPModel(gpytorch.models.ApproximateGP):
    def __init__(self, inducing_points):
        variational_distribution = ❶
        <linearrow /> gpytorch.variational.NaturalVariationalDistribution( ❶
            inducing_points.size(0) ❶
        ) ❶
        variational_strategy = gpytorch.variational.VariationalStrategy( ❶
            self, ❶
            inducing_points, ❶
            variational_distribution, ❶
            learn_inducing_locations=True, ❶
        ) ❶
        super().__init__(variational_strategy)
        self.mean_module = gpytorch.means.ConstantMean()
        self.covar_module = gpytorch.kernels.ScaleKernel(
            gpytorch.kernels.MaternKernel(
                nu=2.5,
                ard_num_dims=inducing_points.shape[1]
            )
        )

    def forward(self, x):
        ... ❷
```

❶ Variational parameters.

❷ Same as the GP.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>